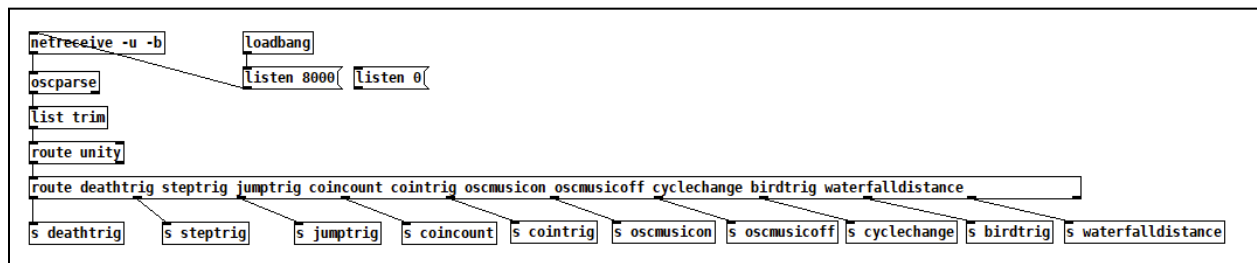


Group Members: Andrew Degan, Ben Nguyen

Pure Data Interaction with Unity

As part of our Final Project, we have sounds that are kept within our Pure Data patch “FinalProjectPatch.pd” as well as certain triggers for these sounds kept within our Unity project. Inside of Pure Data, we utilize a special subpatch entitled “oscReceive,” which receives messages from Unity and triggers events based on the route given.

[illegible]

Each sound has one or more osc triggers that allow for us to input sufficient data and play sounds that accurately reflect what is happening in the Unity project. Inside of our Unity game scripts, we have created functions to send messages depending on triggers within the game.

```

private void SendOscTrigger(string triggerName)
{
    if (!oscInitialized)
    {
        return;
    }

    OSCHandler.Instance.SendMessageToClient("pd", "/unity/" + triggerName, 1);
}

1 reference
private void SendOscInt(string eventName, int value)
{
    if (!oscInitialized)
    {
        return;
    }

    OSCHandler.Instance.SendMessageToClient("pd", "/unity/" + eventName, value);
}

```

For example, if a player is hit by an enemy, it will send the osc message “deathtrig” as well as the necessary “coincount” integer value, which adjusts the sound accordingly.

These messages are all sent from Unity with help from Jorge Garcia Martin’s UnityOSC package, which creates servers at different ports, in this case having Unity send messages (to Pure Data) through port 8000. This way, we are able to add certain commands such as “deathtrig,” “steptrig,” “coincount,” etc. that trigger certain Pure Data functions and sounds upon being called. We are calling these commands mainly inside of our PlayerController.cs and GameManager.cs scripts.

```

// Routine for receiving the OSC
OSCHandler.Instance.UpdateLogs();
Dictionary<string, ServerLog> servers = new Dictionary<string, ServerLog>();
servers = OSCHandler.Instance.Servers;

```

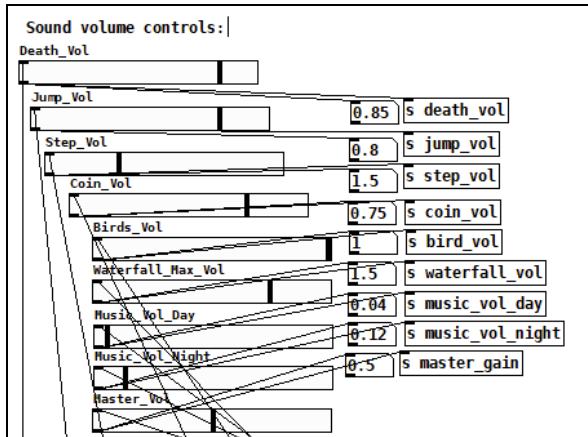
How to Use Pure Data Patch

In order to use our Pure Data patch, you must first find and open up the “FinalProjectPatch.pd” file included inside of our submission folder. Once inside, please make sure that your audio settings are correct (can be found in Media > Audio Settings...) and that sound is coming out of the correct output device.

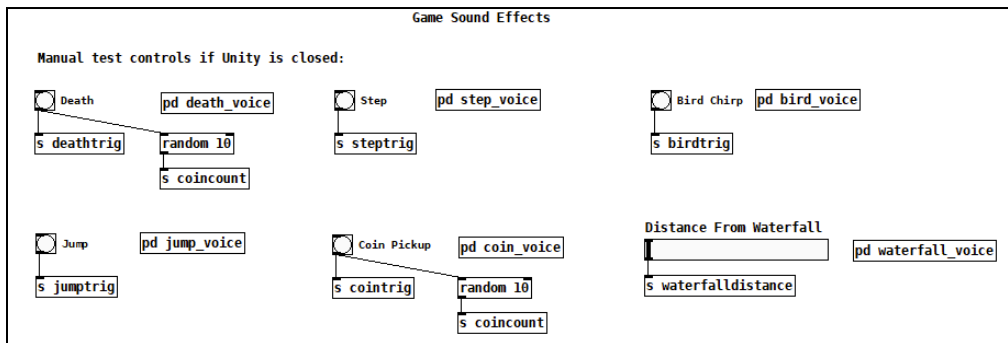
Once you have this completed, please ensure that “Start/Stop DSP” in the top left corner is checked.



Firstly, volume controls are located on the left side of the screen. Please refer to each slider’s label to adjust the correct volume settings. Each individual sound effect and piece of music can be adjusted here. Adjusting the “Master_Vol” will change the overall volume of all sounds.



Next, to the right of the volume controls is a board of our six unique sound effects. Each sound's functions are located within their respective subpatches. While these sounds usually play automatically depending on the circumstances of the Unity game, bangs and sliders have been provided to allow for manual testing. Sounds that are adjusted based on certain integer values (such as coin count) provide a random input value to allow for varied sound results when testing.



Located at the bottom of the patch, we have our Day/Night Generative Music. Included are three buttons: "Start," which starts the music, "Stop," which stops the music, and "Cycle_Flip," which swaps the sound from day to night and vice versa. The graphs to the right side of the buttons each impact the overall sound of the generated music, allowing for custom waveforms, melody scales, and bass scales. These can be adjusted to your liking, but if you would like to return the graphs back to their default settings, please click the "Shared Waveform Table" message located above the buttons.

